

Quinta Lista de Exercícios – Herança e Polimorfismo

1. Herança, Polimorfismo e Métodos Estáticos:

Implementar as classes a seguir:

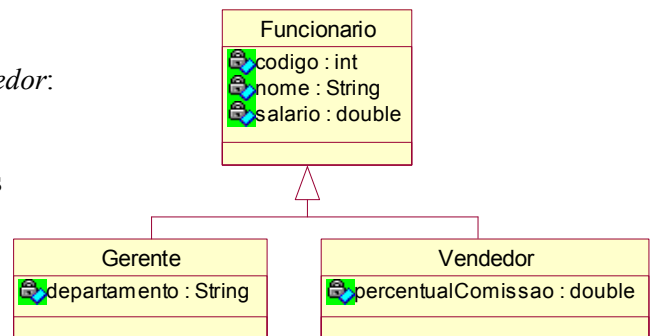
a) Especificações das classes *Funcionário*, *Gerente* e *Vendedor*:

- Atributos:

Numéricos: não podem receber valores menores ou iguais a zero;

Strings: não podem receber valores nulos ou strings vazias.

- Apenas um construtor para cada classe recebendo valores para todos os seus atributos;
- Utilizar a técnica de encapsulamento apresentada no curso e implementar os métodos *get* e *set* para todos os atributos das classes.



b) Classe: CadastroFuncionarios

- Atributos da classe CadastroFuncionarios:

funcionarios (Array de objetos do tipo Funcionario) → não pode manter um array nulo.

quantFuncionarios (inteiro) → Representa a quantidade de funcionários cadastrados e não pode manter um valor menor que zero.

- Implementar apenas um construtor *default* inicializando o atributo *funcionarios* com um array de tamanho 50 e o atributo *quantFuncionarios* com o valor 0.
- Utilizar a técnica de encapsulamento apresentada no curso e implementar apenas os métodos *get* para todos os atributos (ver observação abaixo).
- **OBS:** Não implementar os métodos *set* para os atributos *funcionarios* e *quantFuncionarios* da classe *CadastroFuncionarios*. Os atributos da classe *CadastroFuncionarios* serão manipulados pelos métodos *inserir* e *buscar* detalhados a seguir.
- Métodos da classe CadastroFuncionarios:

boolean inserir(Funcionario func) que insere um objeto do tipo Funcionario no array de funcionários da classe CadastroFuncionarios. Para cada funcionário inserido, o atributo *quantFuncionarios* é incrementado de 1. O método retorna verdadeiro se o funcionário foi inserido com sucesso e falso em uma das três situações seguintes: (1) Quando o array de funcionários estiver cheio; (2) Se o objeto *func* passado como parâmetro é nulo; ou, (3) Se já existe um funcionário cadastrado com o mesmo código.

Funcionario buscar(int cod) que retorna o objeto do tipo Funcionario identificado pelo código informado como parâmetro. Caso o objeto Funcionario não seja encontrado retornar o valor *null*.

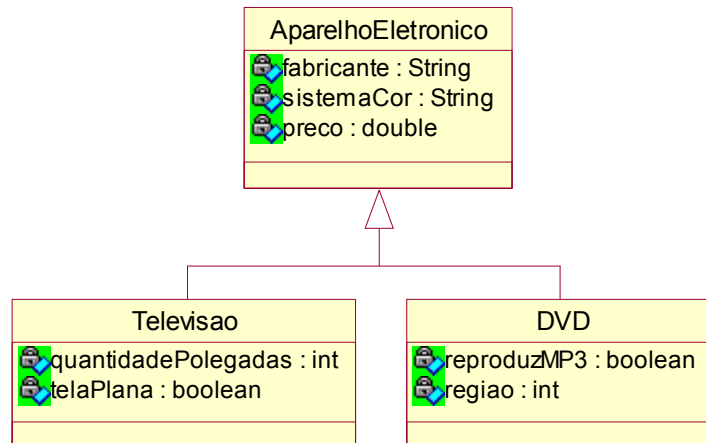
c) Implementar uma aplicação chamada AplicacaoCadastroFuncionarios que cria um objeto do tipo CadastroFuncionarios e oferece um menu com as opções: inserir funcionário, buscar funcionário, listar folha de pagamento e sair. Na opção de menu inserir permita que o usuário decida se ele quer cadastrar um Gerente ou um Vendedor. Na opção de menu listar folha de pagamento, o programa deve chamar um método estático, implementado na classe AplicacaoCadastroFuncionarios, que imprime na tela a folha de pagamento da empresa de acordo com a tabela *exemplo* abaixo:

Folha de pagamento da empresa:

Nome:	Salário:	Função:	
João	R\$ 2000,00	Gerente	– Departamento: Compras
Maria	R\$ 2000,00	Gerente	– Departamento: Vendas
Ana	R\$ 500,00	Vendedor	– Comissão: 5 %
José	R\$ 500,00	Vendedor	– Comissão: 5 %
Total da folha de pagamento: R\$ 5000,00			

Assinatura do método: void imprimeFolha(Funcionario[] func);

2. **Herança e Polimorfismo:** Implementar as classes do modelo abaixo:



a) Atributos:

- Atributos numéricos não podem receber valores menores que zero;
- Atributos do tipo String não podem receber valores nulos ou string vazia;
- O Atributo `sistemaCor` da classe **AparelhoEletronico** não pode receber valores diferentes dos permitidos pelo seu domínio: “NTSC” ou “PAL-M”;
- O Atributo `regiao` da classe **DVD** não pode receber valores diferentes dos permitidos pelo seu domínio: 1, 2, 3 ou 4;

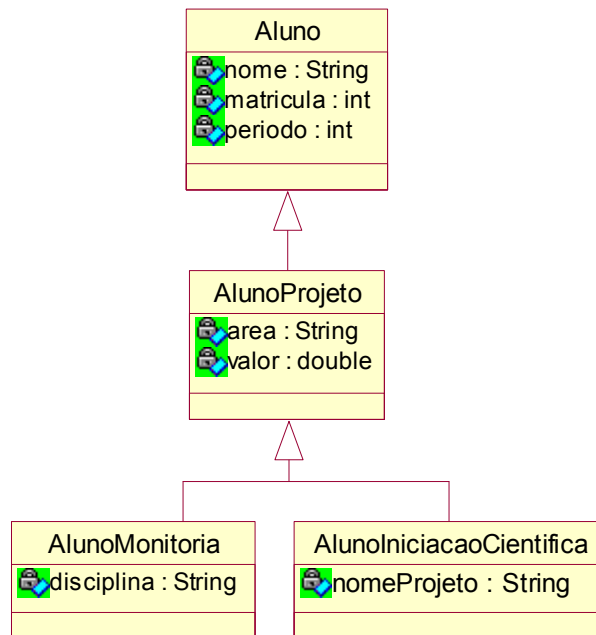
b) Métodos de acesso aos atributos (get / set);

c) Um construtor para cada classe recebendo valores para todos os seus atributos;

d) OBS: Usar a técnica de encapsulamento sugerida no curso;

e) Implementar uma aplicação que cria objetos do tipo **AparelhoEletronico**, a partir de dados fornecidos pelo usuário e, ao final, imprime os valores dos atributos dos objetos criados. Oferecer para o usuário um menu para a escolha entre **Televisão** e **DVD**. OBS: Use uma única variável polimórfica para receber os objetos criados de acordo com a opção do usuário.

3. **Herança e Polimorfismo:** Implementar as classes do modelo abaixo:



a) Atributos:

- Numéricos não podem receber valores menores que zero;
- Strings não podem receber valores nulos ou string vazia.

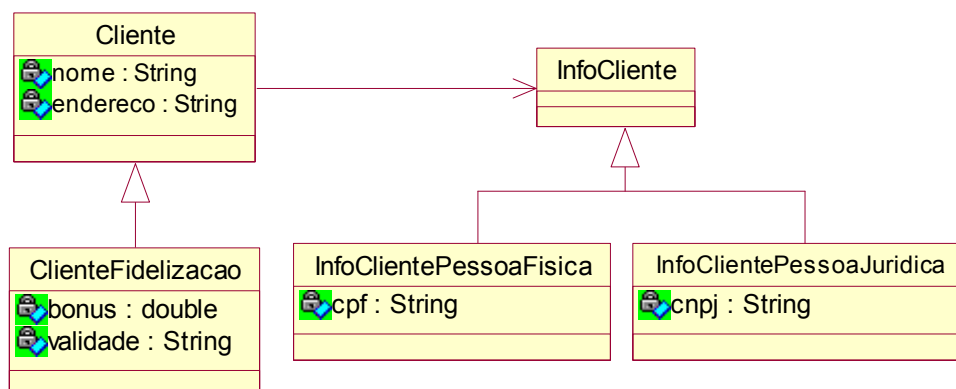
b) Métodos de acesso aos atributos (get / set);

c) Um construtor para cada classe recebendo valores para todos os seus atributos;

d) OBS: Usar a técnica de encapsulamento sugerida no curso;

e) Implementar uma aplicação que cria objetos do tipo **Aluno**, a partir de dados fornecidos pelo usuário e, ao final, imprime os valores dos atributos dos objetos criados. Oferecer para o usuário um menu para a escolha entre **Aluno**, **AlunoMonitoria** e **AlunoIniciacaoCientifica**. OBS: Use uma única variável polimórfica para receber os objetos criados de acordo com a opção do usuário.

4. **Herança e Polimorfismo:** Implementar as classes do modelo abaixo:



a) Atributos:

- Numéricos não podem receber valores menores que zero;

- Strings não podem receber valores nulos ou string vazia.
- b) Métodos de acesso aos atributos (get / set);
- c) Um construtor para cada classe recebendo valores para todos os seus atributos;
- d) OBS: Usar a técnica de encapsulamento sugerida no curso;
- e) Implementar uma aplicação que cria objetos do tipo Cliente, a partir de dados fornecidos pelo usuário e, ao final, imprime os valores dos atributos dos objetos criados. Oferecer para o usuário um menu para a escolha entre Cliente Pessoa Física, Cliente Pessoa Jurídica, Cliente Fidelização Pessoa Física e Cliente Fidelização Pessoa Jurídica. OBS: Use uma única variável polimórfica para receber os objetos criados de acordo com a opção do usuário.

Bom Trabalho!